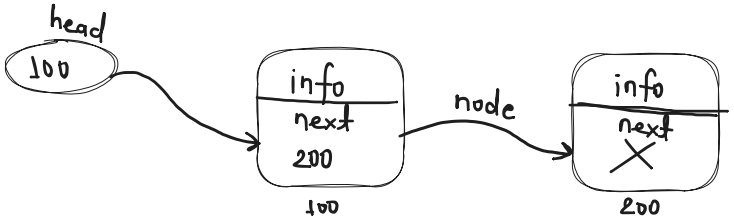
Linkedlist

A linkedlist is a linear data structure, in which elements are stored in separate object called nodes. Each node data and a reference(link) to another node.



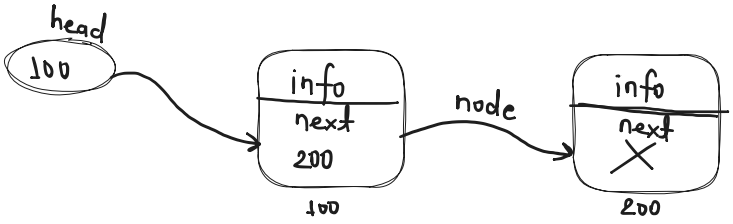
A linkedlist node contains :

- Data : The value stored in the node
- Next : A reference to the next node

--> Type of linkedlist:

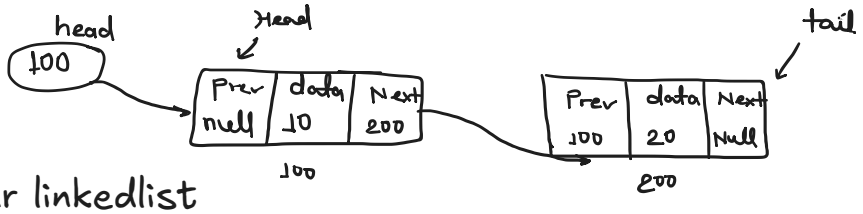
1. Single Linkedlist

Each node points to the next node only.



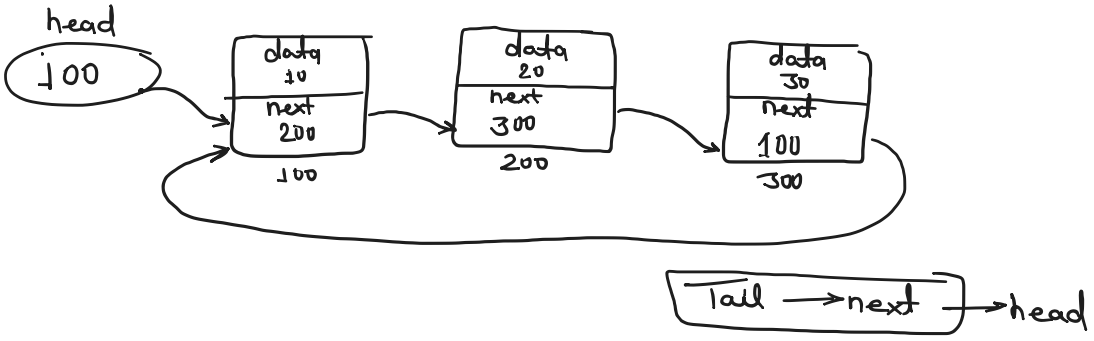
2. Doubly linkedlist

Each node points to both the previous and next nodes.



3. Circular linkedlist

The last node points back the first node instead of null.

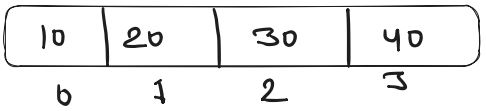


How does linkedlist overcome from the problem of array -

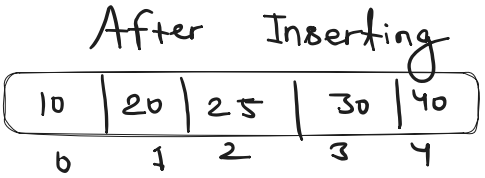
A linkedlist solve some of limitations of array specially when inserting or deleting element frequently.

Problem with array

Array Before Inserting

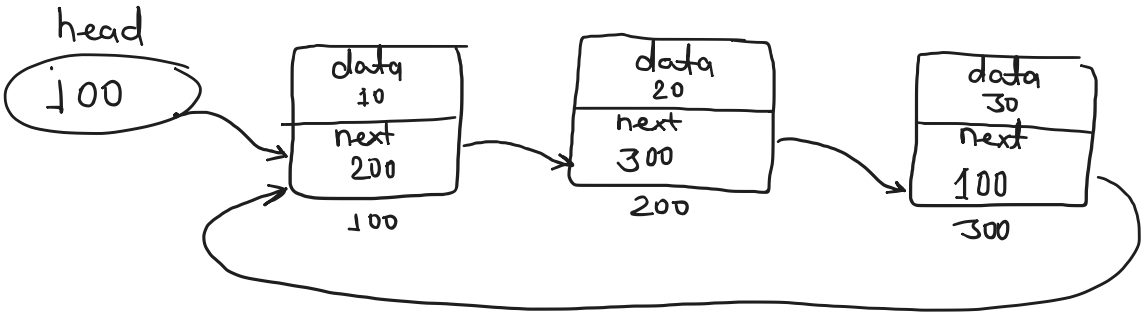


suppose, we want to insert 25 between 20 & 30.



we must shift 30 & 40 to the right in a commercial array. If the array is full, we may also needed to create a larger array and copy its element.

--> A basic Java array has fixed length.
--> Inserting the middle may required shifting element.



Basic Structure of the linkedlist

```
class Node{
    int data;
    Node next;

    Node(int data){
        this.data=data;
        this.next=null;
    }
}
class Main{
    public static void main(String arg[])
       {
        Node first = new Node(10);
        Node second = new Node(20);
        Node third = new Node(30);

    //connect the node
    first.next = second;
    second.next = third;

    //traverse the linkedlist
    Node current = first;
    while(current!=null)
        {
    System.out.println(current.data+"->");
    current = current.next;
        }
    System.out.println("null");
    }
        }
```

Insertion of node in linkedlist at starting or beggining

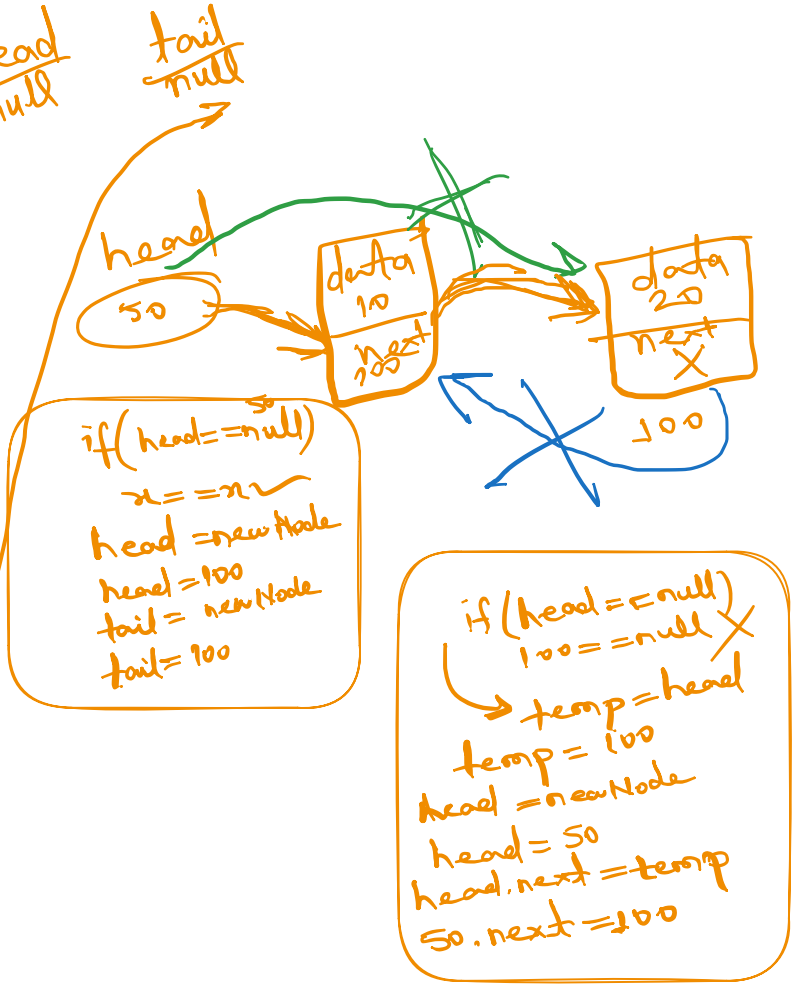
```
class Insert_beg {

    class Node {
      int data;
      Node next;

  public Node(int data) {
     this.data = data;
     this.next = null;
             }
             }

     Node head = null;
     Node tail = null;

 void addAtstart(int data) {
Node newNode = new Node(data);
     if (head == null) {
        head = newNode;
       tail = newNode;
          } else {
      Node temp = head;
       head = newNode;
      head.next = temp;
             }
             }
```

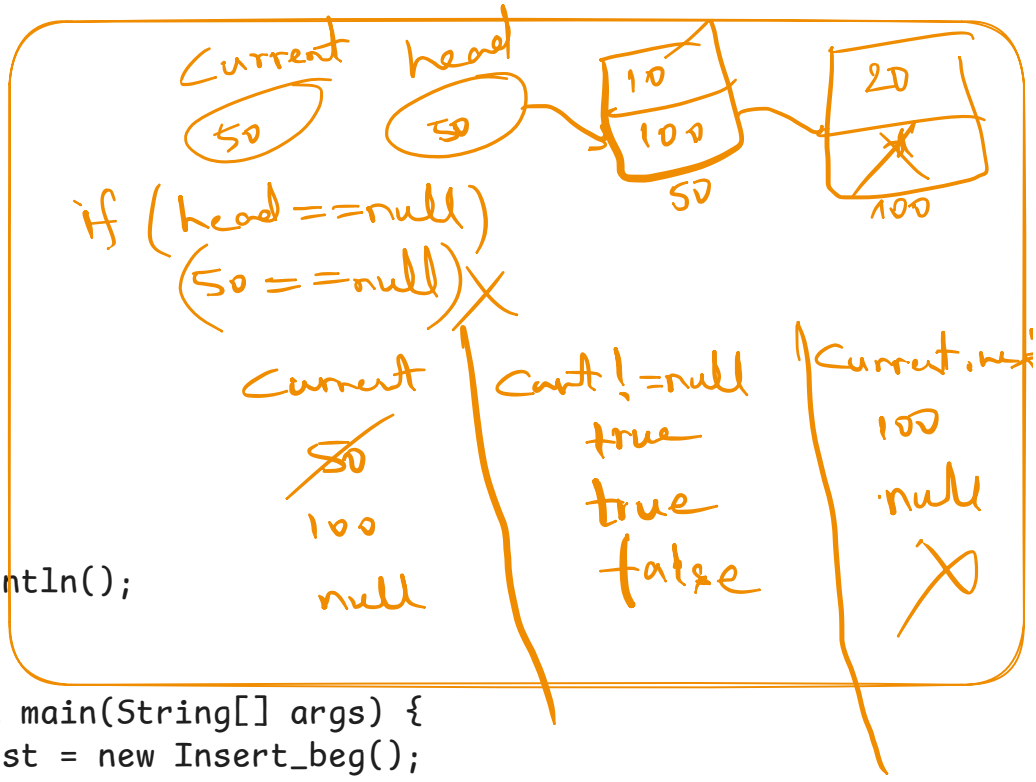


```
void disp(){
        Node current = head;
        if(head==null){
            System.out.println("List is empty");
            return;
        }
         System.out.println("Adding nodes to start of the list");
        while (current!=null) {
            System.out.println(current.data+" ");
            current= current.next;

        }
        System.out.println();
    }

    public static void main(String[] args) {
        Insert_beg sList = new Insert_beg();
        //Adding 1 node
        sList.addAtstart(20);
        sList.disp();

        //Adding 2nd node
        sList.addAtstart(10);
        sList.disp();
    }
}
```



Insertion of data to end of the linkedlist

```
class InsertEnd {
    class Node {
        int data;
        Node next;

        Node(int data) {
        this.data = data;
        this.next = null;
            }
            }

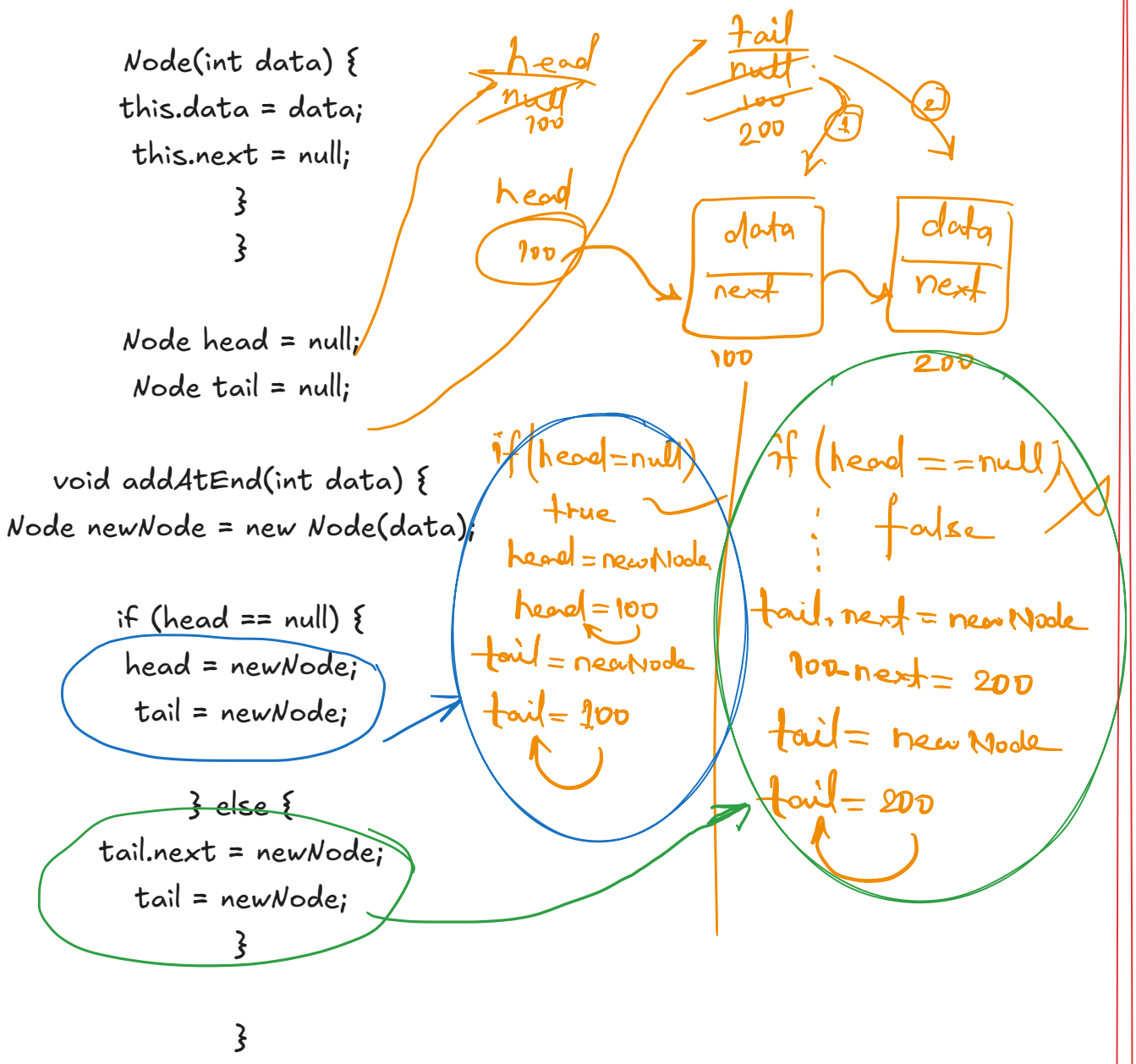
        Node head = null;
        Node tail = null;

    void addAtEnd(int data) {
    Node newNode = new Node(data);

        if (head == null) {
            head = newNode;
            tail = newNode;

        } else {
        tail.next = newNode;
            tail = newNode;
            }

            }
```



```
        void disp() {
    Node current = head;
     if (head == null) {
System.out.println("List is Empty");
        return;
          }


System.out.println("Adding Node to the End of the list");
          while (current != null) {
  System.out.println(current.data + " ");
          current = current.next;
                }
           System.out.println();

                }

    public static void main(String arg[]) {
     InsertEnd slist = new InsertEnd();

          // Add 1 node
       slist.addAtEnd(10);
          slist.disp();

          // Add 2 node
       slist.addAtEnd(20);
          slist.disp();
                }

                }
```